DAY 7 — JWT Authentication & Authorization

Objective

Before Day 7

- Any user could call any API ❌
- No identity ❌
- No access control ❌

Real-world problem: How does the server know WHO is making the request?

Solution we implemented: JWT-based Authentication System

Now: Login → Token → Send Token → Validate → Allow / Deny

- Authentication = identity
- Authorization = permission
- JWT = stateless identity
- Middleware = automatic validation
- Claims = user data

1. Core Concept

JWT = **JSON Web Token**

A **signed identity token** that contains:

- UserId
- Email
- Role
- Expiry
- Issuer

Key idea:

- Server does NOT store session
- Token itself carries identity

This makes system:

- Stateless
- Scalable
- Industry standard

2. Architecture we followed

You implemented **clean architecture correctly**:

Layer	Responsibility
API	Validate token (middleware)
Application	Login flow
Infrastructure	Generate token
Core	User entity

3. JWT Configuration

appsettings.json

- "JwtSettings": {
  "Secret": "...", → signs token (security)
  "Issuer": "...", → who created token
  "Audience": "...", → who can use it
  "ExpiryMinutes": 60 → limits misuse
  }

4. Token Generation (Infrastructure Layer)

JwtTokenService

```
var claims = new List<Claim>
{
   new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
   new Claim(ClaimTypes.Email, user.Email),
   new Claim(ClaimTypes.Role, user.Role)
};
```

- Claims = identity data inside token
- This replaces database lookups in many cases

Token creation:

```
new JwtSecurityToken(...)
```

- Signed using Secret
- Includes expiry
- Encodes user identity

Key insight: Token = "proof that user is authenticated"

5. Dependency Injection

Program.cs

```
builder.Services.AddScoped<IJwtTokenService, JwtTokenService>();
builder.Services.Configure<JwtSettings>(...);
```

- Interface-based → clean architecture

- Loose coupling
- Testable
- Replaceable

6. Authentication Middleware (API Layer)

Program.cs

```
builder.Services.AddAuthentication(...)
.AddJwtBearer(...)
```

What happens internally:

1. Reads token from request header
2. Validates:
   a. Signature
   b. Expiry
   c. Issuer
   d. Audience
3. If valid → attaches user to HttpContext

Important addition:
ClockSkew = TimeSpan.Zero

- Prevents extra 5-minute validity

7. Login Flow Integration

LoginCommandHandler
Before: return token;
After:

```
return new LoginResponse
{
   Token = _jwtTokenService.GenerateToken(user),
   Email = user.Email
};
```

Structured response

- Extensible (add role, id later)
- Cleaner API contract

Important fix: throw new UnauthorizedAccessException(...)

- Correct HTTP semantics (401)

8. DTO Introduction

LoginResponse.cs

- Avoid raw primitives
- Future-proof response
- Matches real APIs

9. Authorization (Protecting Endpoints)

Add [Authorize]

- UsersController
- TestController

Checks if request has valid token
Flow: Request → Middleware → Token valid? → Yes → Controller

10. Swagger JWT Integration (CRITICAL)

Program.cs
Swagger by default doesn't send token, so added

- SecurityDefinition
- SecurityRequirement

Now
Authorize button appears on swagger index
Token gets added to headers

11. Final Working Flow

1. User logs in
2. Server validates credentials
3. JWT token generated
4. Client stores token

5. Client sends token in header
6. Middleware validates token
7. API allows access